

## **Q1. What are the main features of Python? Explain any four features with examples.**

**Answer:**

Introduction:

Python is a high-level, interpreted, general-purpose programming language known for its simplicity and readability. It was created by Guido van Rossum and first released in 1991. Its design philosophy emphasizes code readability with the use of significant indentation.

### **Main Features of Python:**

1. **Simple and Easy to Learn:** Python has a clean syntax similar to English.
2. **Interpreted Language:** Code is executed line by line.
3. **Dynamically Typed:** Variable types are determined at runtime.
4. **Object-Oriented:** Supports classes and objects (OOP concepts).
5. **Free and Open Source:** Freely available to use and distribute.
6. **Extensive Standard Library:** "Batteries included" philosophy.
7. **Portable (Platform Independent):** Runs on Windows, Mac, Linux, etc.

### **Detailed Explanation of Four Features:**

#### **1. Simple and Easy to Learn:**

Python is designed to be easy to read and write. It does not use complex syntax like curly braces {} or semicolons ; for termination. Instead, it relies on indentation to define blocks of code. This makes the code look less cluttered and more like human language.

- **Example:**

Python

```
# A simple Python program is much shorter than C or Java
```

```
print("Hello, World!")
```

#### **2. Interpreted Language:**

Python is an interpreted language, which means the source code is executed line by line, not all at once. The Python Virtual Machine (PVM) reads the code and converts it into machine-readable byte code instantly. This makes debugging easier because the interpreter stops execution as soon as an error is encountered.

- Block Diagram of Execution:

(Diagram Description: Source Code (.py) -> Compiler -> Byte Code (.pyc) -> Interpreter (PVM) -> Machine Code -> Output)

- Example:

If you have 5 lines of code and an error on line 3, Python will execute lines 1 and 2, then stop at line 3.

### 3. Dynamically Typed:

In languages like C++ or Java (Statically Typed), you must declare the data type of a variable (e.g., `int x = 10;`). In Python, you do not need to declare the type. The interpreter automatically detects the type based on the value assigned.

- **Example:**

Python

```
x = 10    # Python knows this is an Integer
```

```
print(type(x))
```

```
x = "Hello" # Now x becomes a String automatically
```

```
print(type(x))
```

### 4. Object-Oriented:

Python supports Object-Oriented Programming (OOP) paradigms. It allows programmers to define classes and create objects to model real-world entities. It supports inheritance, polymorphism, and encapsulation, making it suitable for large software systems.

- **Example:**

Python

```
class Car:
```

```
    def drive(self):
```

```
        print("The car is moving")
```

```
my_car = Car() # Creating an object
```

my\_car.drive()

---

## **Q2. Explain conditional statement with example.**

### **Answer:**

Definition:

Conditional statements (also known as decision-making statements) allow a program to execute a certain block of code only when a specific condition is met. They control the flow of execution based on whether a condition is True or False.

### **Types of Conditional Statements in Python:**

1. if statement
2. if-else statement
3. if-elif-else ladder
4. nested if statement

#### **1. The if Statement:**

This is the simplest form. The code inside the block executes only if the condition is true.

#### **2. The if-else Statement:**

This structure provides an alternative. If the condition is true, the if block executes. If the condition is false, the else block executes.

- Block Diagram / Flowchart:

(Diagram Description: Start -> Condition (Diamond shape) -> True branch -> If Body -> End.  
False branch -> Else Body -> End.)

- **Syntax:**

Python

if condition:

    # code to execute if true

else:

    # code to execute if false

- **Example:**

Python

```
age = 18
```

```
if age >= 18:
```

```
    print("You are eligible to vote.")
```

```
else:
```

```
    print("You are not eligible to vote.")
```

### 3. The if-elif-else Ladder:

Used when checking multiple conditions. The program checks conditions from top to bottom. As soon as one is found true, that block is executed, and the rest are skipped.

- **Example:**

Python

```
marks = 75
```

```
if marks >= 90:
```

```
    print("Grade A")
```

```
elif marks >= 75:
```

```
    print("Grade B")
```

```
elif marks >= 50:
```

```
    print("Grade C")
```

```
else:
```

```
    print("Fail")
```

---

**Q3. Explain while loop with example.**

**Answer:**

Definition:

A while loop is an entry-controlled loop. It repeatedly executes a block of statements as long as a given condition remains True. The condition is checked before the execution of the loop body. If the condition is false initially, the loop body will not execute even once.

**Key Components:**

1. **Initialization:** Setting the loop variable.
2. **Condition:** The criteria to keep the loop running.
3. **Update:** Incrementing or decrementing the variable to eventually terminate the loop.

Block Diagram / Flowchart of While Loop:

(Diagram Description: Start -> Condition (Diamond) -> If True -> Execute Loop Body -> Go back to Condition. If False -> Exit Loop -> End.)

**Syntax:**

Python

while condition:

    # statements

    # update statement

**Example 1: Printing numbers from 1 to 5**

Python

```
i = 1      # Initialization
```

```
while i <= 5: # Condition
```

```
    print(i)
```

```
    i = i + 1 # Update (Increment)
```

*Output:*

Plaintext

1

2

3

4

5

Example 2: Using a while loop with user input

This is commonly used when you don't know how many times the loop needs to run.

Python

```
active = True
```

```
while active:
```

```
    message = input("Type 'quit' to exit: ")
```

```
    if message == 'quit':
```

```
        active = False
```

```
    else:
```

```
        print(f"You typed: {message}")
```

Infinite Loop Warning:

If you forget the update statement (e.g., `i = i + 1`), the condition might never become False, causing an "Infinite Loop" which can crash the program.

---

**Q4. What is type conversion? Differentiate between implicit and explicit conversion with examples.**

**Answer:**

Definition:

Type conversion (or Type Casting) is the process of converting the value of one data type (integer, string, float, etc.) into another data type. Python provides two types of type conversion.

**1. Implicit Type Conversion:**

- **Definition:** Python automatically converts one data type to another without any user intervention. This usually happens when operations involve mixed types (e.g., Integer + Float).
- **Mechanism:** Python promotes the smaller data type to the larger data type to prevent data loss.
- **Example:**

Python

```
num_int = 10    # Integer
```

```
num_float = 2.5 # Float
```

```
# Python automatically converts num_int to float to perform addition
```

```
result = num_int + num_float
```

```
print("Value:", result)
```

```
print("Type:", type(result))
```

*Output:* Value: 12.5, Type: <class 'float'>

## 2. Explicit Type Conversion (Type Casting):

- **Definition:** The user manually converts a data type into another using built-in functions like int(), float(), str(), etc.
- **Mechanism:** This is required when Python cannot automatically convert types (e.g., adding a String number to an Integer).
- **Example:**

Python

```
num_string = "100"
```

```
num_int = 20
```

```
# result = num_string + num_int <-- This would cause an Error
```

```
# Explicitly converting string to int  
num_string_converted = int(num_string)
```

```
result = num_string_converted + num_int  
print("Result:", result)
```

*Output:* Result: 120

#### Differences between Implicit and Explicit Conversion:

| Feature       | Implicit Conversion                     | Explicit Conversion                                         |
|---------------|-----------------------------------------|-------------------------------------------------------------|
| User Action   | Not required (Automatic).               | Required (Manual).                                          |
| Function Used | None (Interpreter handles it).          | int(), float(), str(), etc.                                 |
| Data Loss     | Minimal risk (promotes to larger type). | High risk (e.g., converting float 3.9 to int 3 loses data). |
| Readability   | High readability.                       | Can make code verbose.                                      |

---

#### Q5. Explain operator and type of operators with example.

**Answer:**

Definition:

An operator is a special symbol in Python used to perform operations on variables and values. The values on which the operator acts are called operands.

- *Example:* In  $a + b$ ,  $+$  is the operator, and  $a$  and  $b$  are operands.

Types of Operators:

Python generally divides operators into the following groups:

1. Arithmetic Operators:

Used for mathematical calculations.



- + (Add), - (Sub), \* (Mul), / (Div), % (Modulus), \*\* (Exponentiation), // (Floor Div).
- *Example:* 10 // 3 results in 3.

## 2. Relational (Comparison) Operators:

Used to compare values. Returns True or False.

- == (Equal), != (Not Equal), >, <, >=, <=.
- *Example:* 5 > 3 returns True.

## 3. Logical Operators:

Used to combine conditional statements.

- and (Returns True if both are true).
- or (Returns True if one is true).
- not (Reverses the result).
- *Example:* (5 > 3) and (2 < 4) returns True.

## 4. Assignment Operators:

Used to assign values to variables.

- =, +=, -=, \*=.
- *Example:* x += 5 is same as x = x + 5.

## 5. Membership Operators:

Used to test if a sequence is presented in an object.

- in, not in.
- *Example:*

Python

```
list1 = [1, 2, 3]
```

```
print(1 in list1) # Returns True
```

## 6. Identity Operators:

Used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location.

- is, is not.

Block Diagram of Operator Categories:

(Diagram Description: A central block "Python Operators" branching out to Arithmetic, Logical, Comparison, Assignment, Bitwise, Membership, Identity)

### **Comprehensive Example Program:**

Python

a = 10

b = 3

# Arithmetic

print("Addition:", a + b)    # 13

# Comparison

print("Is a greater than b?", a > b) # True

# Logical

print("Is a > 5 AND b < 5?", a > 5 and b < 5) # True

# Assignment

c = a

c += 2

print("Value of c:", c) # 12